POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY 1



INFORMATION SYSTEM ANALYSIS AND DESIGN

E-COMMERCE MARKETPLACE

Requirements Identification and Requirements Analysis

Multi-Vendor E-Commerce Specialization

Student	Chu Tuyet Nhi
Student ID	B23DCCE075
Class	E23TTNT02-B
Lecturer	PGS.TS. Tran Dinh Que

Hanoi, October 2026

Contents

1 Requirements Identification **1**
1.1 Introduction . . . 1
1.2 Business Background . . . 1
1.3 Stakeholder Analysis . . . 2
1.4 Business Activity and Information Analysis . . . 2
1.5 Current Problems . . . 3
1.6 Proposed Information System . . . 4
1.7 Stakeholder Needs . . . 5
1.8 Functional Requirements . . . 6
1.9 Non-Functional Requirements . . . 8
1.10 Requirement Prioritization . . . 9

2 Requirements Analysis **10**
2.1 Actor Analysis . . . 10
2.2 Use Case Model . . . 10
2.3 Use Case Specifications . . . 12
2.4 Supporting Processes and Transaction States . . . 23
2.5 Domain Concepts . . . 24
2.6 Cart, Order, and Checkout Analysis . . . 25
2.7 Analysis Class Model . . . 26
2.8 Business Rules . . . 29

1. Requirements Identification

1.1. Introduction

E-Commerce Marketplace is a multi-vendor online retail platform that connects consumers with independent third-party merchants selling physical goods. Unlike a single-merchant online store where one enterprise controls the catalogue, warehouse stock, and order fulfilment under a single roof, a multi-vendor marketplace coordinates independent shops that set their own prices, maintain their own inventory counts, and pack their own parcels. Every purchase therefore crosses organizational boundaries among the buyer, the selling shop, internal platform staff, the payment gateway, and the shipping carrier.

This report investigates the operational structure of E-Commerce Marketplace and establishes the requirements and analysis models for its information system. Chapter 1 examines the business model, stakeholder relationships, current operational bottlenecks, proposed system boundary, and functional and non-functional requirements. Chapter 2 develops the actor and use case models, detailed use case specifications, activity workflows, transaction state lifecycles, domain concepts, analysis class diagrams, and governing business rules.

1.2. Business Background

1.2.1. Organization and Business Model

E-Commerce Marketplace operates as a commercial intermediary rather than a traditional retailer. Instead of buying stock from wholesalers and storing goods in a centralized warehouse, the platform provides the digital infrastructure where independent Sellers register their own Shops, publish product listings, manage stock availability, and prepare confirmed customer orders for carrier pickup. Internally, daily platform operations are divided among four staff roles: the Administrator, Support Staff, Finance Staff, and the Manager. Externally, the platform integrates with a Payment Service for online transactions and a Shipping Provider for physical parcel delivery and cash-on-delivery (COD) collection.

Revenue is generated through transaction fees charged on completed orders. Because the marketplace collects online payments or tracks COD collection on behalf of participating shops before disbursing net earnings, financial accounting distinguishes the gross merchandise value (GMV) of ordered items, the actual funds collected from the customer, the platform commission fee, and the net payout owed to the Seller. Keeping these figures distinct allows staff to reconcile customer refunds, unpaid COD orders, and seller settlements accurately.

1.2.2. Products, Customers, and Core Business Activities

The marketplace catalogue consists of everyday consumer products sold as discrete physical units. At the domain level, each Product record has a platform-wide identifier (productId) and represents a specific sellable listing and stock-keeping unit (sku) owned by a single Shop. Because SKU codes are assigned by individual merchants, SKU uniqueness is enforced within each Shop through the composite key (shopId, sku). If two shops sell the same commercial item, each shop maintains its own Product record with its own price, description, and stock counts. Customers browse the catalogue by category and price or search for specific products by keyword, voice, or image. Before committing to a purchase, a buyer needs clear visibility into which shop is selling the item, whether stock is available, and what the final payable total will be.

Operations across the marketplace flow through eleven connected activities: seller shop registration, catalogue listing moderation, product discovery, shopping cart management, single-shop checkout, payment processing, seller order packing, carrier shipping and delivery, post-purchase support and refunds, seller financial settlement, and management reporting. A core distinction in this workflow separates shopping intention from purchase commitment. Adding items to a cart records a buyer's temporary selection across one or more shops without holding inventory, whereas completing checkout creates a binding order for one selected shop and reserves the corresponding stock.

1.3. Stakeholder Analysis

Operating a multi-vendor marketplace requires balancing the expectations of four internal staff roles and five external participant groups. Customers expect accurate stock counts, transparent pricing, and reliable order tracking, while Sellers require timely listing approval, protection against overselling, and predictable payouts once parcels are delivered. Internally, the Administrator safeguards catalogue quality, Support Staff mediate post-purchase disputes using verifiable order evidence, Finance Staff reconcile payment and payout ledgers, and the Manager evaluates overall marketplace performance. The roles and primary information needs of all nine stakeholder groups are detailed in Table 1.1.

Table 1.1: Stakeholders, roles, and main interests

Stakeholder	**Role**	**Main Interest or Need**
Administrator	Approves shops, moderates listings, and manages accounts	Complete shop applications, pending product submissions, and moderation history
Support Staff	Resolves customer complaints and refund requests	Fixed order snapshots, payment statuses, tracking logs, and seller responses
Finance Staff	Reconciles customer funds, refunds, and seller payouts	Verified collection references, refund deductions, platform fees, and net entitlements
Manager	Monitors marketplace operations and commercial performance	Period reports separating merchandise value, collected funds, refunds, fees, and payouts
Customer	Browses products, places orders, tracks delivery, and requests support	Selling shop identity, live availability, confirmed order total, and delivery progress
Seller	Operates an approved shop, manages stock, and fulfils orders	Moderation decisions, own-shop order details, stock commitments, and payout statements
Payment Service	Processes online payment requests and refund transactions	Transaction amount, order reference code, and requested payment action
Shipping Provider	Picks up parcels, updates tracking, and collects COD cash	Shop pickup address, recipient contact, delivery address, and shipment reference
Seller Supplier	Supplies physical inventory to individual sellers outside the platform	External purchase orders arranged directly by each seller

1.4. Business Activity and Information Analysis

In the existing manual and spreadsheet-based operation, transaction data is fragmented across disconnected files maintained by different teams:

- **Catalogue moderation and customer profiles:** Sellers submit new product listings and price changes through shared spreadsheets, while the Administrator records approval decisions in a separate moderation log. Because listing edits and approval states are not linked in a single database, unapproved price changes or discontinued items frequently remain visible to shoppers. Similarly, customer delivery addresses are copied manually from profile sheets into order logs. When a buyer updates their default profile address for a future purchase, staff inspecting the profile sheet can mistakenly overwrite or misread the delivery address agreed for an earlier order still in transit.
- **Order handling and inventory control:** When a buyer submits an order, staff manually split multi-shop selections into separate shop sheets and forward them to the respective Sellers. Stock counts are deducted only when warehouse staff finish packing at the end of a shift rather than at the moment of checkout. Consequently, two buyers can check out the last remaining unit of an item simultaneously, forcing the Seller to cancel one of the confirmed orders. Conversely, when a buyer requests a refund, staff sometimes add the refunded quantity back into available stock immediately upon approving the request, before the Seller has received and inspected the returned parcel.

- **Payment verification, shipping, and financial reporting:** Online payment notifications from the Payment Service and COD collection reports from the Shipping Provider are recorded in a single paid-or-unpaid spreadsheet column. This simplified label cannot distinguish whether an online payment timed out with an unknown outcome, whether COD cash has been collected by the courier, whether a customer refund has been issued, or whether the Seller has already been paid. At the end of the reporting period, staff sum raw order totals as platform revenue without deducting cancelled orders, customer refunds, or seller payouts.

The core information records produced and consumed across the marketplace are organized in Table 1.2, which distinguishes live operational records from historical transaction snapshots.

Table 1.2: Information catalogue of E-Commerce Marketplace

Information	**Produced By**	**Used By**	**Purpose**
Account and address	Customer, Seller, Admin	Customer, Seller, Admin	Authentication, role access control, and default delivery address
Shop application	Seller, Administrator	Seller, Administrator	Seller onboarding, shop pickup address, and approval history
Product and stock	Seller, Administrator	Customer, Seller, Admin	Catalogue browsing, search filtering, and live and committed stock control
Cart and cart items	Customer	Customer	Temporary holding of selected items and quantities grouped by Shop
Order and order items	Confirmed checkout	Customer, Seller, Staff	Fulfilment instruction and fixed price, item, and address snapshots
Payment record	Payment Service, Finance	Customer, Seller, Finance	Online payment verification, COD collection flag, and reconciliation
Shipment record	Seller, Shipping Provider	Customer, Seller, Support	Parcel handover recording, carrier tracking code, and delivery timestamps
Support and refund	Customer, Seller, Staff	Customer, Seller, Staff	Post-purchase dispute investigation, return inspection, and full-order refund
Settlement record	Finance Staff	Seller, Finance, Manager	Calculation of gross item value, platform fee, refund deduction, and net payout
Reviews and reports	Customer, Staff	All authorized roles	Verified buyer ratings and period-based operational and financial evaluation

1.5. Current Problems

The operational gaps described above reinforce one another across the order lifecycle. When catalogue moderation and inventory updates lag behind live customer traffic, buyers place orders on stale data. When payment, shipping, and support records lack shared reference keys, staff cannot resolve exceptions without manual cross-checking across multiple spreadsheets. Six primary operational problems (P1 to P6) and their underlying causes and business consequences are identified in Table 1.3.

Table 1.3: Problem analysis

Problem	Possible Cause	Consequence
P1: Inconsistent product listings	Listing edits and moderation decisions are maintained in separate files	Customers see unapproved or inactive items with unclear shop ownership
P2: Outdated stock and overselling	Inventory is deducted after packing rather than committed at checkout	Multiple customers can purchase the final unit, causing seller cancellations
P3: Unclear order and delivery status	Order, packing, and carrier tracking updates are scattered across separate logs	Customers face delays while staff manually cross-check files to answer enquiries
P4: Confused payment and payout records	Online payment timeouts, COD collection, refunds, and payouts share one status column	Customers risk duplicate charges and sellers risk premature or inaccurate payouts
P5: Untraceable support and refunds	Support notes are not linked to order snapshots, buyer evidence, or return checks	Disputes are handled inconsistently and returned goods are restocked before inspection
P6: Inaccurate sales and settlement reports	Manual reports sum gross order totals without separating cancellations, refunds, or fees	The Manager and Sellers rely on distorted figures when evaluating performance

1.6. Proposed Information System

1.6.1. System Purpose and Scope

The proposed E-Commerce Marketplace Information System replaces fragmented spreadsheets with a unified transactional platform that coordinates shops, moderated listings, inventory commitments, single-shop orders, payments, shipments, support cases, refunds, and settlements:

- **Single-shop ownership and SKU listings:** Each Seller registers and operates at most one Shop (with a recorded pickup address), while the marketplace hosts many independent Shops. Every Product record has a global productId and a shop-scoped SKU code (shopId, sku) with its own unit price, available stock, and committed stock.
- **Multi-shop cart and single-shop checkout:** A Customer can collect products from several shops inside one shopping cart, where lines are grouped visually by selling Shop. To keep fulfilment, shipping, and settlement unambiguous, each checkout transaction creates one Order for one selected Shop group while leaving items from other shops intact in the cart. Dynamic shipping fee calculation is outside the current scope, so the order total equals the sum of confirmed item amounts ($\sum \text{quantity} \times \text{unitPrice}$).
- **Payment, fulfilment, cancellation, and full-order refunds:** Checkout supports both online payment via the Payment Service and COD collected by the Shipping Provider. Each confirmed Order is fulfilled by its owning Shop and shipped as a single parcel. Customers may cancel an order directly only before the Seller hands the parcel over to the carrier. After handover, issues are handled through formal support cases, and approved refunds are processed as full-order refunds of the collected total.
- **Financial settlement and multi-modal discovery:** Each Order links to at most one seller settlement record once delivery and payment collection are verified. For product discovery, core keyword, category, and price filtering are complemented by optional voice and image search.

1.6.2. System Boundary and Context Diagram

Inside the software boundary lie account management, shop onboarding, listing moderation, product search, cart management, single-shop checkout, stock reservation, order and shipment tracking, payment and COD status recording, support case resolution, full-order refunds, seller settlement, buyer reviews, and period reporting. Outside the software boundary lie the external Payment Service and Shipping Provider platforms, physical warehouse picking and packing performed by Sellers, external supplier procurement, and the human stakeholders interacting with the system.

Figure 1.1 presents the stakeholder and system context diagram of the E-Commerce Marketplace Information System, showing the central system boundary and the directional data flows exchanged with each internal and external participant.

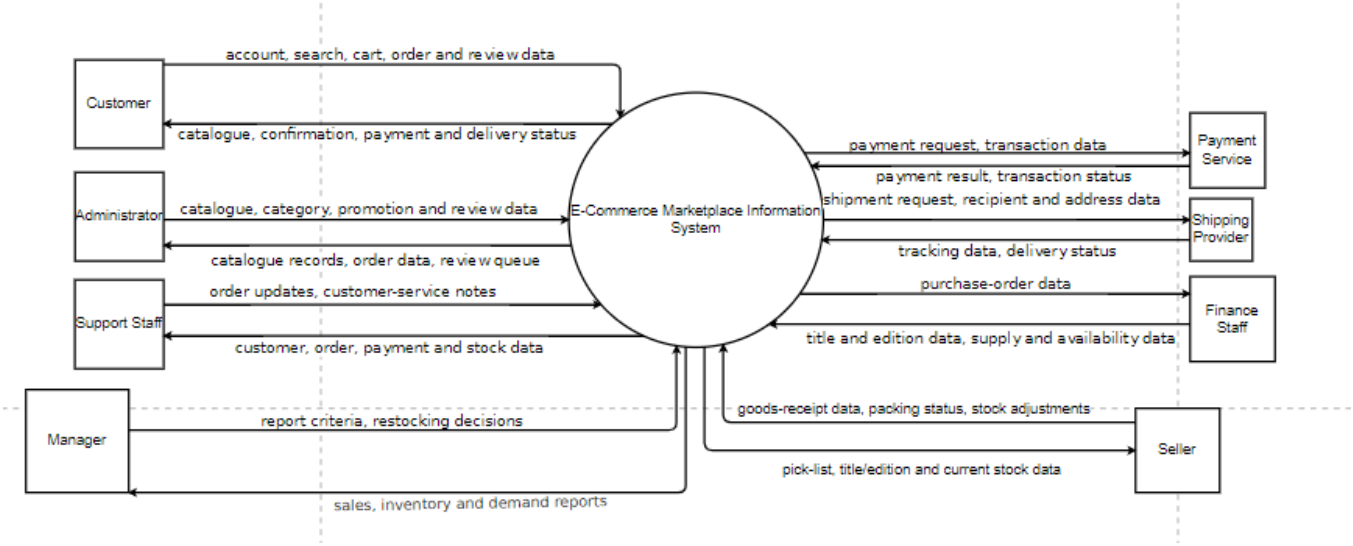


Figure 1.1: Stakeholder and system context diagram of the E-Commerce Marketplace Information System.

As shown in Figure 1.1, every stakeholder interaction is mediated through structured information flows:

- **Customer and Seller flows:** The Customer submits account credentials, search queries, cart selections, checkout confirmations, support requests, and product reviews to the system, and receives catalogue listings, order confirmations, and separate payment and delivery statuses. On the supply side, the Seller submits shop applications, SKU listings, stock adjustments, and carrier handover updates, and receives moderation status notifications, own-shop orders, and settlement statements.
- **Internal governance flows:** The Administrator submits shop approval, listing moderation, and account decisions while receiving shop applications, pending SKU listings, and the moderation queue. Support Staff receive customer, order, payment, tracking, and evidence data to evaluate disputes and submit dispute decisions, refund approvals, and support notes. Finance Staff receive order totals, payment references, and payout ledger data and submit unknown payment reconciliations, refund executions, and settlement records, while the Manager provides reporting period and filter criteria and receives GMV, collected funds, refund, fee, and settlement reports.
- **External service flows:** The system sends payment requests, refund requests, and transaction references to the Payment Service and receives verified payment results, refund outcomes, and transaction statuses. For physical logistics, the system transmits shipment requests, shop pickup addresses, and recipient address snapshots to the Shipping Provider and receives tracking updates, parcel delivery statuses, and COD collection confirmations.

1.6.3. System Objectives

To resolve problems P1 through P6, the proposed system enforces six operational objectives: (1) expose only administrator-approved product listings belonging to active shops, (2) recheck live availability and reserve stock at checkout so two buyers cannot claim the same unit, (3) link every order to its owning shop, payment reference, and carrier tracking code, (4) maintain independent state lifecycles for order progress, online payment verification, COD cash collection, customer refunds, and seller payouts, (5) attach every support case and full-order refund to fixed order snapshots while requiring seller confirmation of returned goods before restocking, and (6) generate role-scoped reports that distinguish merchandise value, cancellations, collected funds, refunds, platform fees, and net seller payouts.

1.7. Stakeholder Needs

Translating the system context into concrete requirements requires examining how concurrent operations affect each stakeholder. In a live marketplace, two buyers frequently view the final unit of a popular item at the same time. If stock is held as soon as an item is placed in a cart, abandoned carts quickly

lock up a seller's inventory. Conversely, if stock is deducted only after warehouse packing, concurrent checkouts inevitably cause overselling. Moving stock from availableStock to committedStock at checkout confirmation resolves both risks while preventing a Seller from manually reducing available stock below quantities already promised to active orders.

A similar coordination challenge arises when an online payment request times out without a definitive response from the Payment Service. Treating an unanswered timeout as a failed payment and allowing an immediate retry risks charging the buyer twice if the initial request actually succeeded at the gateway. Recording the payment in an explicit Unknown state keeps the order and its committed stock on hold while Finance Staff reconcile the provider reference. The operational needs of each stakeholder group are paired with their business drivers in Table 1.4.

Table 1.4: Stakeholder needs

Stakeholder	**Need**	**Business Reason**
Customer	Browse and search only purchasable products with clear shop identity, price, and stock	Inconsistent listings
Customer	Search the catalogue using keywords, filters, spoken queries, or an uploaded image	Multi-modal discovery
Customer	Reconfirm live price, available stock, delivery address, and payable total at checkout	Outdated stock
Customer	Inspect separate statuses and timestamps for order processing, payment, and delivery	Unclear order status
Customer	Submit a post-delivery support or refund claim with evidence and view the decision	Untraceable disputes
Seller	Create and update own-shop product listings and view moderation feedback	Inconsistent listings
Seller	View availableStock and committedStock and record SKU adjustments with audit reasons	Outdated stock
Seller	Pack eligible own-shop orders, record carrier handover, and reply to support cases	Manual order handling
Seller	Review own-shop sales summaries and per-order settlement calculations	Inaccurate reports
Administrator	Evaluate shop applications and product submissions and record approval or rejection reasons	Publication control
Support Staff	Inspect fixed order snapshots, tracking history, buyer evidence, and seller responses	Untraceable disputes
Finance Staff	Reconcile unknown online payments and track collections, refunds, and payouts separately	Confused money states
Manager	Analyze period reports separating merchandise value, collected funds, refunds, and fees	Inaccurate reports
Payment Service	Receive exact transaction amounts and unique references and return verified outcomes	Confused money states
Shipping Provider	Receive shop pickup address, recipient contact, address snapshot, and shipment reference	Manual shipping logs

1.8. Functional Requirements

Twenty-eight functional requirements (FR01 to FR28) govern account and shop onboarding, catalogue moderation and discovery, shopping cart and checkout, order fulfilment and inventory, and after-sales support, settlement, and reporting, as listed in Table 1.5. Priorities follow the MoSCoW scheme, where Must Have denotes foundational capabilities required for core transaction integrity, Should Have denotes important operational and governance workflows, and Could Have denotes optional convenience features.

Table 1.5: Functional requirements

ID	Category	Requirement	Priority
FR01	Account	The system shall register Customer and Seller accounts and validate required identity and contact details.	Must Have
FR02	Account	The system shall authenticate users and restrict actions and data views to their assigned roles and owned records.	Must Have
FR03	Account	The system shall save Customer profile and defaultAddress edits without modifying address snapshots on existing orders.	Must Have
FR04	Shop	The system shall allow a Seller to submit an application (including pickupAddress) for at most one Shop and view the status.	Must Have
FR05	Shop	The system shall allow the Administrator to approve or reject a shop application with a recorded moderationReason.	Must Have
FR06	Catalogue	The system shall allow a Seller to create, edit, or discontinue product listings within their own approved Shop.	Must Have
FR07	Catalogue	The system shall allow the Administrator to approve or reject listings with a rejectionReason and expose only published items of active Shops.	Must Have
FR08	Search	The system shall search published products by keyword, category, and price range or return an explicit no-match message.	Must Have
FR09	Discovery	The system shall display product name, SKU, selling Shop, unit price, description, stock status, and verified buyer reviews.	Must Have
FR10	Cart	The system shall add, update, or remove cart items using positive integer quantities grouped by selling Shop.	Must Have
FR11	Cart	The system shall calculate line amounts and shop subtotals using current product prices without holding stock.	Must Have
FR12	Checkout	The system shall check out one selected Shop group per order while retaining items from other Shops in the cart.	Must Have
FR13	Checkout	The system shall recheck Shop status, listing status, availableStock, unit price, and delivery address before order acceptance and require buyer confirmation if a price changed.	Must Have
FR14	Order	The system shall create one Order and its OrderItems storing fixed SKU, product name, unit price, and delivery address snapshots.	Must Have
FR15	Stock	The system shall move purchased SKU quantities from availableStock to committedStock once upon order acceptance and release them once upon eligible pre-handover cancellation.	Must Have
FR16	Payment	The system shall record payment method, amount, provider reference, codCollected flag, and updatedAt timestamp, holding unknown online payment results for reconciliation before retry.	Must Have
FR17	Fulfilment	The system shall allow a Seller to confirm packing and record carrier handover for eligible orders belonging to their own Shop.	Must Have
FR18	Order	The system shall allow a Customer to cancel an order directly only before carrier handover and record the cancellation outcome.	Must Have
FR19	Shipping	The system shall record carrier tracking codes, delivery status transitions, updatedAt timestamps, and COD cash collection confirmations by reference.	Must Have
FR20	Order	The system shall display separate order, payment, and shipment statuses with timestamps to the ordering Customer, the owning Seller, and authorized staff.	Must Have

ID	Category	Requirement	Priority
FR21	Inventory	The system shall allow a Seller to record stock adjustments for own-shop SKUs with a reason while rejecting negative availability or reductions that violate committedStock.	Must Have
FR22	Support	The system shall allow a Customer to submit a post-purchase support or refund request linked to an order with a reason and evidence.	Should Have
FR23	Refund	The system shall record the Seller response (sellerResponse), Support Staff decision, physical return inspection flag (returnInspected), and execution of an approved full-order refund.	Should Have
FR24	Settlement	The system shall allow Finance Staff to record gross item value, platform fee, refund deduction, net seller entitlement, payout status, and settledAt timestamp per order.	Should Have
FR25	Reporting	The system shall generate role-scoped period reports distinguishing merchandise value, cancellations, refunds, collected funds, fees, and seller payouts.	Should Have
FR26	Review	The system shall accept at most one rating and review comment from the ordering Customer for each delivered OrderItem.	Should Have
FR27	Search	The system shall transcribe a spoken query into editable text and execute search only after Customer confirmation.	Could Have
FR28	Search	The system shall retrieve candidate published listings matching an uploaded product image for Customer comparison.	Could Have

1.9. Non-Functional Requirements

To ensure that functional workflows remain dependable under real marketplace traffic, the system must satisfy twelve non-functional requirements (NFR01 to NFR12) spanning performance, security, usability, availability, reliability, scalability, maintainability, and transactional data integrity:

- **Search response performance (NFR01):** The system shall return keyword and filter search results within 3 seconds at the 95th percentile under a baseline load of 10,000 published listings and 50 concurrent users, and within 5 seconds at the 95th percentile under 20,000 listings and 100 concurrent users. *Should Have.*
- **Role and ownership access control (NFR02):** The system shall enforce role-based and ownership-based authorization so that a Customer can access only their own cart and orders, a Seller can manage only their own Shop, SKUs, and orders, and staff can invoke only functions assigned to their role. *Must Have.*
- **Credential and payment data protection (NFR03):** The system shall store account passwords exclusively as salted cryptographic hashes and shall never store raw payment card numbers or security codes inside marketplace tables. *Must Have.*
- **Usability and validation feedback (NFR04):** The system shall support full keyboard navigation across search, cart, and checkout forms and display field-level validation messages beside any rejected input. *Should Have.*
- **Cross-device browser compatibility (NFR05):** The system shall render search, cart, checkout, and order tracking interfaces without horizontal clipping on standard desktop (1366×768) and mobile (390×844) viewports in current Chrome and Edge browsers. *Should Have.*
- **Operational availability and graceful degradation (NFR06):** The system shall maintain at least 99% uptime during scheduled operating windows (08:00 to 17:00 across five business days per week) and keep keyword search, cart management, and COD checkout operational if voice recognition, image matching, or the external online payment gateway is temporarily unreachable. *Should Have.*
- **Transactional durability and recovery (NFR07):** The system shall persist accepted orders, stock commitments, and payment records in durable storage so that no committed transaction is lost across a service restart. *Must Have.*

- **Idempotent callback handling (NFR08):** The system shall process repeated checkout submissions, payment callbacks, or carrier tracking updates carrying the same reference code without duplicating charges, refunds, or stock changes. *Must Have.*
- **Catalogue and traffic scalability (NFR09):** The system shall scale from 10,000 to 20,000 published listings and from 50 to 100 concurrent users within the 5-second response ceiling of NFR01 without requiring schema changes to existing records. *Should Have.*
- **Modular integration maintainability (NFR10):** The system shall isolate external payment and shipping gateway adapters behind clear service boundaries so that updating a provider interface does not alter core order, inventory, or settlement rules. *Should Have.*
- **Stock commitment integrity (NFR11):** The system shall guarantee non-negative availableStock and committedStock across all SKUs and prevent concurrent checkouts or manual seller edits from overcommitting the final available unit. *Must Have.*
- **Order snapshot immutability and audit trail (NFR12):** The system shall preserve confirmed order item and address snapshots against subsequent catalogue or profile edits and log the acting user, timestamp, and reason for every moderation, stock adjustment, support, refund, and settlement action. *Must Have.*

1.10. Requirement Prioritization

Grouping the requirements by delivery priority establishes a practical implementation sequence:

- **Must Have:** Core account registration, single-shop onboarding, listing moderation, keyword search, multi-shop cart management, single-shop checkout with stock reservation and fixed snapshots, online and COD payment recording, carrier tracking, and manual stock control (FR01 to FR21, together with NFR02, NFR03, NFR07, NFR08, NFR11, and NFR12) form the minimum viable transactional baseline. Without these safeguards, the platform cannot prevent overselling, cross-shop order mix-ups, or duplicate payment charges.
- **Should Have:** Post-handover support case management, full-order refunds, seller settlement accounting, period reporting, verified buyer reviews, and quantitative service-level targets (FR22 to FR26, alongside NFR01, NFR04 to NFR06, NFR09, and NFR10) complete the commercial and governance lifecycle once core ordering is stable.
- **Could Have:** Voice-based query transcription and image-based product discovery (FR27 and FR28) enrich buyer convenience without affecting core transaction processing if schedule or computing resources are constrained.
- **Won't Have (Out of Scope):** Combined multi-shop orders in a single checkout, dynamic shipping fee calculation, partial line-item refunds, split shipments per order, platform-owned warehouses, automated supplier purchase orders, promotional voucher engines, and internal digital wallets are excluded from the current system scope.

2. Requirements Analysis

2.1. Actor Analysis

In Unified Modeling Language (UML) analysis, an actor represents a cohesive role played by a human user or an external system when interacting with the subject boundary. Based on the stakeholder context established in Chapter 1, the marketplace interacts with five primary actor categories:

- **Customer:** Represents both unauthenticated visitors who browse and search public listings and authenticated buyers who manage their shopping cart, complete single-shop checkout, make payments, track order progress, cancel eligible orders before carrier handover, raise post-purchase support claims, and submit product reviews.
- **Seller:** Represents an authenticated merchant who registers at most one Shop, maintains product listings and SKU stock levels for that Shop, packs confirmed orders and hands parcels to the carrier, responds to customer support cases, and inspects own-shop sales and settlement statements.
- **Platform Staff (Administrator, Support Staff, Finance Staff, and Manager):** Represents the internal governance roles of the marketplace. The Administrator reviews shop applications and moderates product listings, Support Staff investigate post-handover complaints and approve full-order refunds, Finance Staff reconcile unknown online payments and record seller settlements, and the Manager generates operational and financial performance reports.
- **Payment Service:** Represents the external payment gateway that processes online payment requests and refund instructions by transaction reference and returns verified outcomes.
- **Shipping Provider:** Represents the external logistics carrier that receives parcel pickup and recipient details and returns tracking milestones, delivery confirmations, and COD cash collection updates.

2.2. Use Case Model

2.2.1. Actor and Use Case Identification

Visitors access public browsing and product search under the Customer role, whereas saving cart modifications, completing checkout, and inspecting personal order records require authentication. Seller operations are scoped strictly to the Seller's own approved Shop, and internal governance operations are partitioned across the Administrator, Support Staff, Finance Staff, and Manager roles. All eighteen use cases (UC01 to UC18), their participating actors, and their traced functional requirements are presented in Table 2.1.

Table 2.1: Use case catalogue

ID	Use Case	Primary Actor	Supporting Actors	Requirements
UC01	Manage Account	Customer, Seller, Admin	None	FR01 to FR03
UC02	Register Shop	Seller	None	FR04
UC03	Approve Shop	Administrator	None	FR05
UC04	Manage Products	Seller	None	FR06
UC05	Moderate Listings	Administrator	None	FR07
UC06	Search Products	Customer	None	FR08, FR27, FR28
UC07	View Product	Customer	None	FR09
UC08	Manage Cart	Customer	None	FR10, FR11
UC09	Checkout	Customer	None	FR12 to FR16
UC10	Make Payment	Customer	Payment Service, Finance	FR16
UC11	Process Orders	Seller	Shipping Provider	FR17
UC12	Cancel Order	Customer	None	FR15, FR18
UC13	Track Order	Customer, Seller	Shipping, Payment Services	FR19, FR20
UC14	Manage Inventory	Seller	None	FR21

ID	Use Case	Primary Actor	Supporting Actors	Requirements
UC15	Resolve Support Case	Customer, Support Staff	Seller, Finance, Providers	FR22, FR23
UC16	Manage Settlement	Finance Staff	Seller	FR24
UC17	Generate Reports	Manager, Admin, Seller	None	FR25
UC18	Submit Review	Customer	None	FR26

2.2.2. UML Use Case Diagram and Relationships

Figure 2.1 presents the UML use case diagram of the E-Commerce Marketplace Information System, linking the eight actors across the left and right boundaries to the eighteen functional use cases and three specialized search modalities.

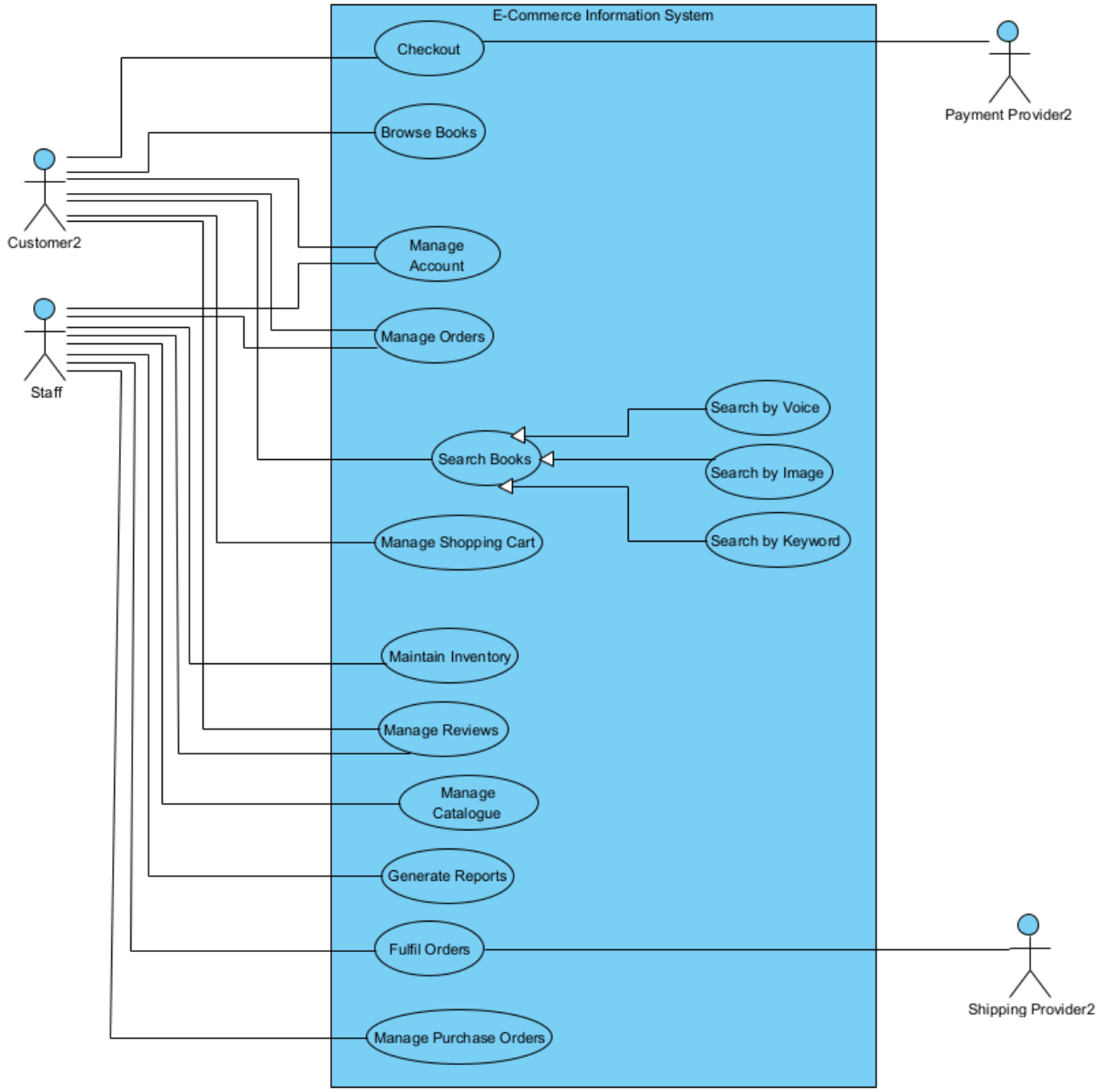


Figure 2.1: UML use case diagram of E-Commerce Marketplace.

In Figure 2.1, the high-level use case diagram summarizes the primary user goals and external provider interactions across the system boundary:

- **Search specialization:** *Search by Keyword*, *Search by Voice*, and *Search by Image* specialize the product search goal through UML generalization arrows, allowing buyers to enter queries by text, voice, or image while sharing the same publication filter rules.

- **Checkout and external provider links:** *Checkout* connects the buyer to the external payment provider when online payment is selected, whereas COD orders complete checkout with a pending collection obligation. Fulfilment and order tracking connect with the external shipping provider to record parcel handover and delivery updates.

2.3. Use Case Specifications

The dynamic behavior of six primary operational use cases (*Search Products*, *Manage Cart*, *Checkout*, *Track Order*, *Manage Products*, and *Manage Inventory*) is specified below through structured scenario tables and UML activity diagrams.

2.3.1. Search Products

Table 2.2: Search Products specification

Field	**Specification**
Use Case Name	Search Products (UC06)
Primary Actor	Customer
Goal	The Customer locates published products matching their criteria and inspects the selling shop, unit price, and availability before selecting an item.
Trigger	The Customer enters a search query or applies category or price filters on the storefront.
Preconditions	The public product catalogue is online and accessible.
Postconditions	Matching published products belonging to active shops, or an explicit no-match message, are displayed. Cart contents and inventory counts remain unchanged.
Main Success Scenario	1. The Customer enters search criteria (keyword, category selection, or price range) or confirms a transcribed voice or image query. 2. The system validates that the query parameters and price bounds are well-formed. 3. The system queries the catalogue for published products belonging to active shops that satisfy the criteria. 4. The system returns the matching product cards showing product name, selling shop, unit price, and stock availability. 5. The Customer browses the returned list and selects a product to inspect its detail page.
Alternative Flows	1. Search by Voice: The Customer grants microphone access and speaks a query. The system transcribes the audio into editable text and waits for Customer confirmation before executing step 2. 2. Search by Image: The Customer uploads a JPEG or PNG image. The system extracts visual features and returns candidate published products with shop names and prices for comparison. 3. Invalid filter input: If the query is blank or the minimum price exceeds the maximum price, the system highlights the invalid input and prompts the Customer to revise the criteria. 4. No matching products: If no published listing from an active shop matches the criteria, the system displays a clear no-match message and allows the Customer to broaden the filters. 5. Listing state change before viewing: If a product becomes out of stock or unpublished between search display and item selection, the detail view shows its updated unavailable state and blocks checkout.
Exceptions	1. If microphone permission is denied or voice or image recognition fails, the system notifies the Customer and falls back cleanly to keyword search. 2. If the catalogue search index experiences a temporary timeout, the system displays a service error message with a retry option rather than misreporting that no products exist.

Field	Specification
Business Rules	BR01: Only published products belonging to active shops are eligible to appear in customer search results. Spoken queries require explicit Customer confirmation prior to search execution.
Priority	Must Have for keyword, category, and price filtering. Could Have for voice and image search.

Figure 2.2 illustrates the validation retry loop for malformed filter inputs and the separate terminal paths for matched published products versus empty result sets.

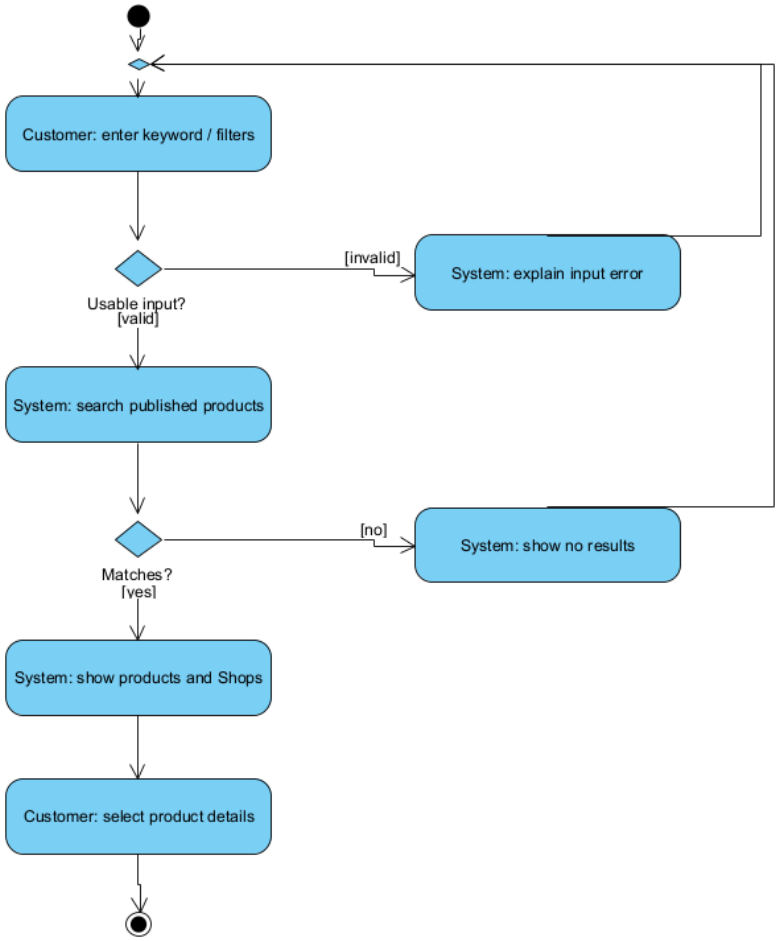


Figure 2.2: Search Products activity diagram.

2.3.2. Manage Cart

Table 2.3: Manage Cart specification

Field	Specification
Use Case Name	Manage Cart (UC08)
Primary Actor	Customer
Goal	The Customer reviews and modifies selected products, quantities, and shop-grouped subtotals in their shopping cart prior to checkout.
Trigger	The Customer opens the shopping cart, adds a product from the catalogue, updates an item quantity, or removes a cart line.
Preconditions	The Customer is authenticated when saving persistent cart modifications, and a target product is selected when adding a new line.
Postconditions	Validated cart changes and recalculated shop subtotals are saved without holding inventory. Invalid quantity edits leave the prior valid cart state untouched.

Field	Specification
Main Success Scenario	1. The Customer opens the shopping cart and selects a cart action (add item, update quantity, or remove item). 2. For an add or quantity update action, the system checks that the target product is published, its selling shop is active, and the requested positive integer quantity does not exceed current availableStock. 3. The system adds or updates the CartItem in the Customer's cart, or deletes the CartItem if removal was chosen. 4. The system groups all active cart lines by selling Shop and recalculates each line amount and shop subtotal using live product unit prices. 5. The system displays the updated cart view to the Customer, indicating that cart items do not hold stock until checkout.
Alternative Flows	1. Remove item: When the Customer chooses to remove a line, the system deletes the CartItem directly and refreshes the remaining shop subtotals. 2. Unavailable product or excessive quantity: If the requested quantity is not a positive integer, exceeds availableStock, or targets an inactive listing, the system displays the currently available quantity and retains the previous valid cart state. 3. Previously saved item becomes unavailable: If a listing already in the cart is later discontinued by the Seller or suspended by the Administrator, the system marks that line as unavailable and prevents it from being selected for checkout until removed. 4. Live price update: If a Seller updates a product's unit price while the item sits in a cart, reopening the cart recalculates the subtotal using the latest live price so the Customer sees current figures before starting checkout.
Exceptions	1. If a storage error prevents saving the cart update, the system alerts the Customer and reloads the last confirmed cart state. 2. Any attempt to read or modify a cart belonging to another Customer account is rejected by access control.
Business Rules	BR01, BR02, and BR03: Cart quantities must be positive integers, cart lines are grouped by selling Shop, and cart modifications do not reserve stock.
Priority	Must Have.

Figure 2.3 shows how the system verifies the requested quantity against live stock before saving the CartItem under its selling Shop and refreshing the shop subtotal without holding inventory.

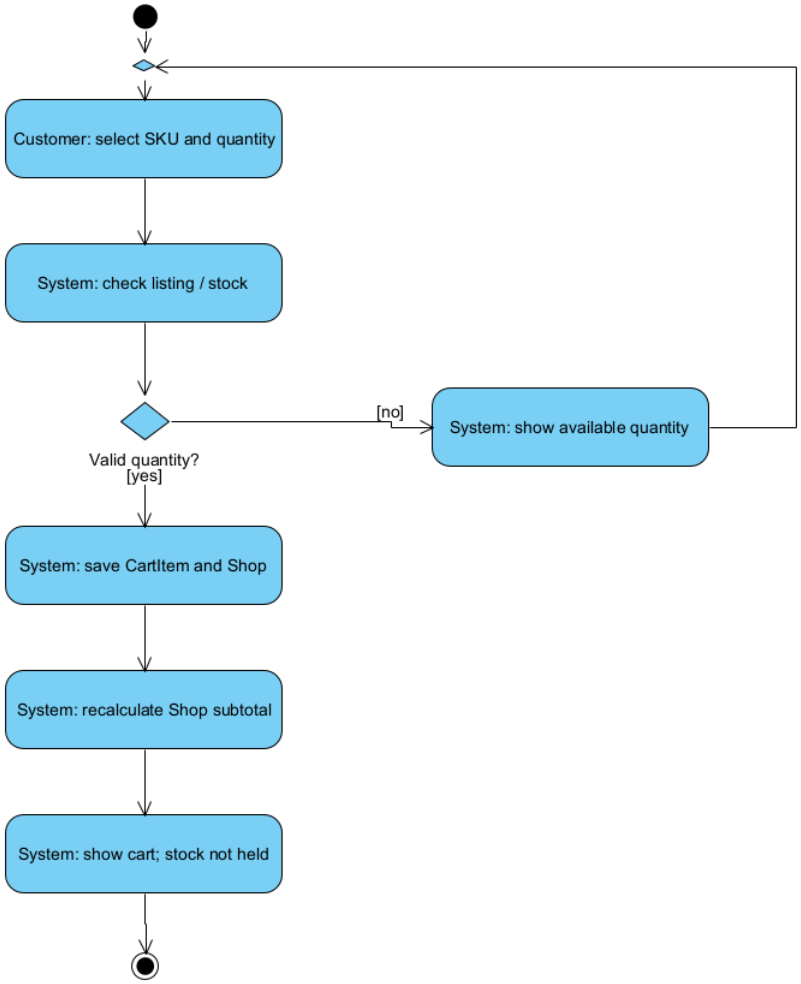


Figure 2.3: Manage Cart activity diagram.

2.3.3. Checkout

Table 2.4: Checkout specification

Field	Specification
Use Case Name	Checkout (UC09)
Primary Actor	Customer
Supporting Actors	Payment Service (invoked when the online payment branch is selected).
Goal	The Customer converts one selected shop group from their cart into a confirmed single-shop Order with locked snapshots and reserved inventory.
Trigger	The Customer selects one shop group in the cart and initiates checkout.
Preconditions	The Customer is logged in and the selected shop group contains at least one valid cart item.
Postconditions	A new Order and its OrderItems are persisted for the selected Shop with fixed item and address snapshots, purchased quantities are moved from availableStock to committedStock, and only that shop's items are removed from the cart. If online payment fails, the pending order is cancelled and its committed stock is released.

Field	Specification
Main Success Scenario	1. The Customer selects a single shop group from the cart, enters or confirms the delivery address, and chooses a payment method (Online Payment or COD). 2. The system validates that the selected shop is active, all items remain published with sufficient live stock, and the delivery address is complete. 3. The system calculates and displays the payable order total for Customer confirmation. 4. Upon confirmation, the system rechecks live stock and prices, creates the Order and OrderItem records with fixed SKU, name, unit price, and address snapshots, and moves the purchased quantities from availableStock to committedStock. 5. If the Customer selected COD, the system sets the payment status to PendingCollection and marks the order as Placed. 6. If the Customer selected Online Payment, the system sends a payment request to the Payment Service, receives a verified Paid result, and marks the order as Placed. 7. The system removes the checked-out shop's items from the cart and displays the order confirmation to the Customer.
Alternative Flows	1. Validation or stock failure: If any item in the selected shop group has insufficient stock, an inactive listing, or an incomplete delivery address, the system displays the required corrections and aborts order creation without partial stock deduction. 2. Mid-checkout price change: If a Seller updates an item's live price before order commitment, the system halts acceptance, displays the revised total, and requires explicit Customer reconfirmation. 3. Online payment failure: If the Payment Service returns a definitive Failed result, the system records the failed payment attempt, cancels the pending order, releases the committed stock back to availableStock, and notifies the Customer. 4. Online payment timeout (Unknown state): If the payment gateway times out without a clear result, the system records the payment status as Unknown, keeps the order in AwaitingPayment with stock committed, and blocks duplicate payment retries until Finance Staff reconcile the reference. 5. Late payment arrival after cancellation: If a verified payment callback arrives for an order that has already been cancelled, the system logs the collection and routes the record to Finance Staff for a full-order refund.
Exceptions	1. If a database fault prevents saving the Order, OrderItems, and stock commitment together, the transaction rolls back completely so no phantom stock deduction remains. 2. Duplicate checkout submissions carrying the same request reference return the existing order without creating a second order or deducting stock twice.
Business Rules	BR03, BR04, BR05, BR06, BR07, and BR08: Every order belongs to one Shop, locks item and address snapshots, commits stock at acceptance, and follows distinct online and COD payment rules.
Priority	Must Have.

Figure 2.4 contrasts the direct COD acceptance path with the online payment gateway interaction, the Unknown timeout reconciliation loop, and the Failed payment stock rollback.

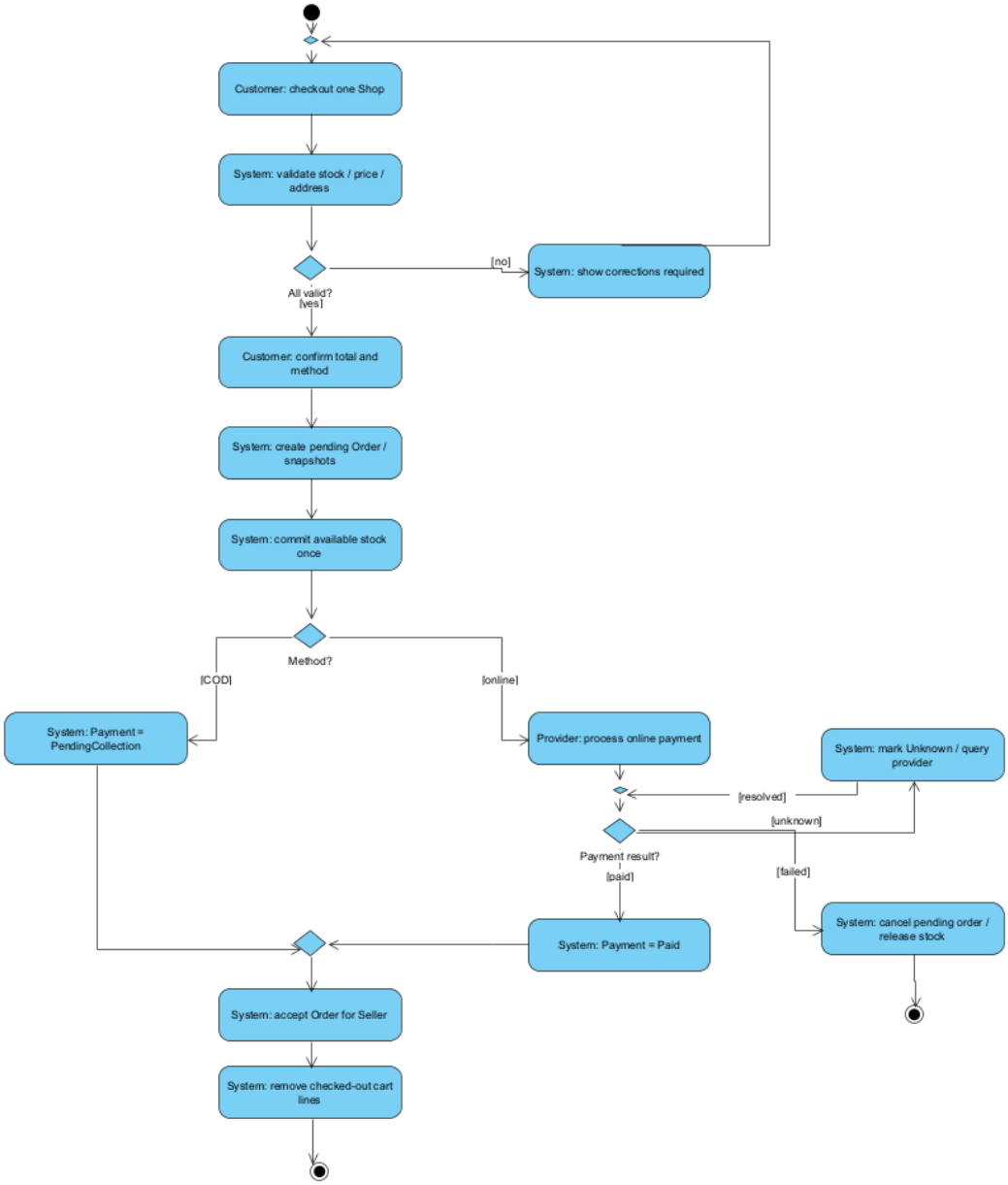


Figure 2.4: Checkout activity diagram.

2.3.4. Track Order

Table 2.5: Track Order specification

Field	Specification
Use Case Name	Track Order (UC13)
Primary Actor	Customer or Seller
Supporting Actors	Shipping Provider and Payment Service.
Goal	An authorized Customer or Seller inspects the current, separated statuses and timestamps of an order, its payment collection, and its parcel shipment.
Trigger	A Customer selects an order from their purchase history or a Seller selects an order assigned to their shop.
Preconditions	The user is logged in and selects a valid order reference.
Postconditions	The order snapshot details and separate order, payment, and shipment statuses with updatedAt timestamps are displayed to the authorized user. No transaction state is modified if external providers have no new updates.

Field	Specification
Main Success Scenario	1. The Customer or Seller selects an order to view its tracking details. 2. The system verifies that the requesting user is either the Customer who placed the order or the Seller who owns the fulfilling Shop (or authorized platform staff). 3. The system loads the Order record together with its linked Payment and Shipment records. 4. The system queries the latest carrier delivery status from the Shipping Provider and the latest payment status from the Payment Service by reference code. 5. The system presents a consolidated tracking view that displays the order status, payment status (including codCollected), and parcel shipment status as three distinct indicators with their updatedAt timestamps. 6. The user reviews the tracking information and completes the inquiry.
Alternative Flows	1. Unauthorized access attempt: If a user requests an order that belongs neither to their Customer account nor to their Seller Shop, the system denies access and displays an authorization error without leaking order details. 2. Pre-handover order state: If the Seller has not yet handed the parcel to the Shipping Provider, the system displays the current order and payment statuses and indicates that carrier tracking has not yet commenced. 3. External provider unreachable: If the Shipping Provider or Payment Service does not respond during a status refresh, the system retains the last confirmed status and updatedAt timestamp rather than overwriting state. 4. COD delivery prior to cash remittance: When the carrier marks a COD parcel as Delivered before cash collection is confirmed, the shipment status shows Delivered while codCollected remains false and payment status remains PendingCollection. 5. Payment under reconciliation: If an online payment is in the Unknown state, the tracking view informs the Customer that Finance Staff reconciliation is in progress and prevents duplicate payment submissions.
Exceptions	1. If a temporary storage error prevents retrieving the order record, the system displays an error notification rather than rendering partial or inconsistent status flags.
Business Rules	BR08, BR10, and BR14: Order visibility is restricted to the owning buyer, owning seller, and authorized staff, and parcel delivery, COD cash collection, and seller payout are tracked as independent events.
Priority	Must Have.

Figure 2.5 shows how *Track Order* verifies order ownership before reading the order, payment, and shipment records and either incorporating a new provider update or retaining the last known timestamp.

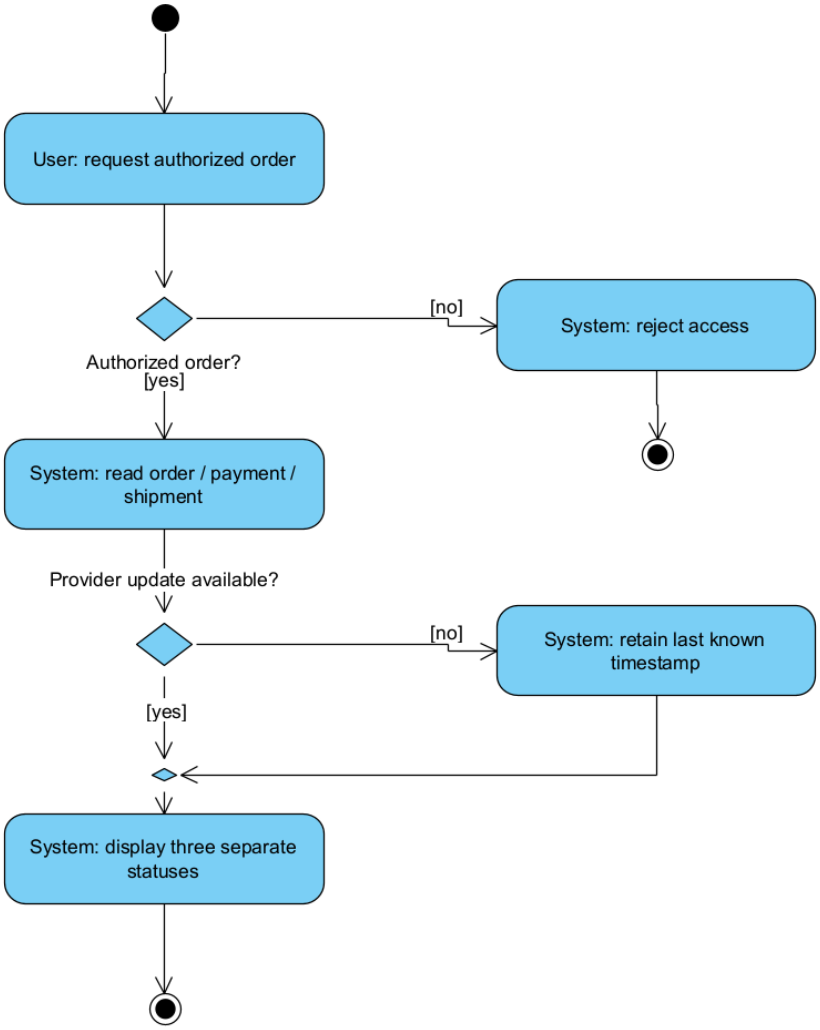


Figure 2.5: Track Order activity diagram.

2.3.5. Manage Products

Table 2.6: Manage Products specification

Field	Specification
Use Case Name	Manage Products (UC04)
Primary Actor	Seller
Supporting Actors	Administrator (reviews pending submissions via Moderate Listings).
Goal	The Seller creates, edits, or discontinues product listings within their own Shop while enforcing moderation rules and preserving historical order snapshots.
Trigger	The Seller opens the shop catalogue management view and selects a product management action.
Preconditions	The Seller is logged in and owns an approved, active Shop.
Postconditions	Valid new or edited listings are assigned a productId, saved as PendingReview, and published only upon Administrator approval, while discontinued listings are archived from public sale without altering existing OrderItem snapshots.

Field	Specification
Main Success Scenario	1. The Seller selects an action in their Shop catalogue (create new listing, edit existing listing, or discontinue listing). 2. For creating or editing a listing, the Seller submits the shop-scoped SKU code, product name, category, description, unit price, and initial stock count. 3. The system verifies shop ownership, checks that all mandatory fields are present, confirms that the unit price is positive, and verifies that the composite key (shopId, sku) is unique within the Shop. 4. The system saves the listing in a PendingReview state and routes it to the Administrator moderation queue. 5. The Administrator inspects the listing details and records an approval decision. 6. The system updates the listing status to Published, makes the product visible in public search results while the Shop is active, and notifies the Seller.
Alternative Flows	1. Discontinue listing: When the Seller chooses to discontinue an existing product, the system archives the listing so it no longer appears in search or new checkouts, while keeping the Product record linked to historical OrderItems. 2. Invalid listing data or duplicate SKU: If required fields are blank, the unit price is zero or negative, or a new SKU code duplicates an existing (shopId, sku) pair in the same Shop, the system rejects the submission and prompts the Seller to correct the invalid fields. 3. Moderation rejection: If the Administrator rejects a submitted listing, the system saves the rejectionReason on the Product record, marks the listing as Rejected, keeps it hidden from buyers, and displays the feedback to the Seller for revision. 4. Attempt to overwrite committed stock: If the Seller attempts to use listing edits to reduce stock on an active SKU below committedStock, the system directs the Seller to the dedicated Manage Inventory workflow.
Exceptions	1. If a storage failure occurs while saving the listing or moderation decision, the system rolls back the update so unapproved changes are never exposed to shoppers.
Business Rules	BR01 and BR06: Every Product belongs to one Shop under a unique (shopId, sku) pair, requires Administrator approval prior to publication, and leaves historical order snapshots untouched when edited or discontinued.
Priority	Must Have.

In Figure 2.6, product submissions pass through shop-scoped SKU validation into the PendingReview state before the Administrator either publishes the Shop listing or records a rejection reason.

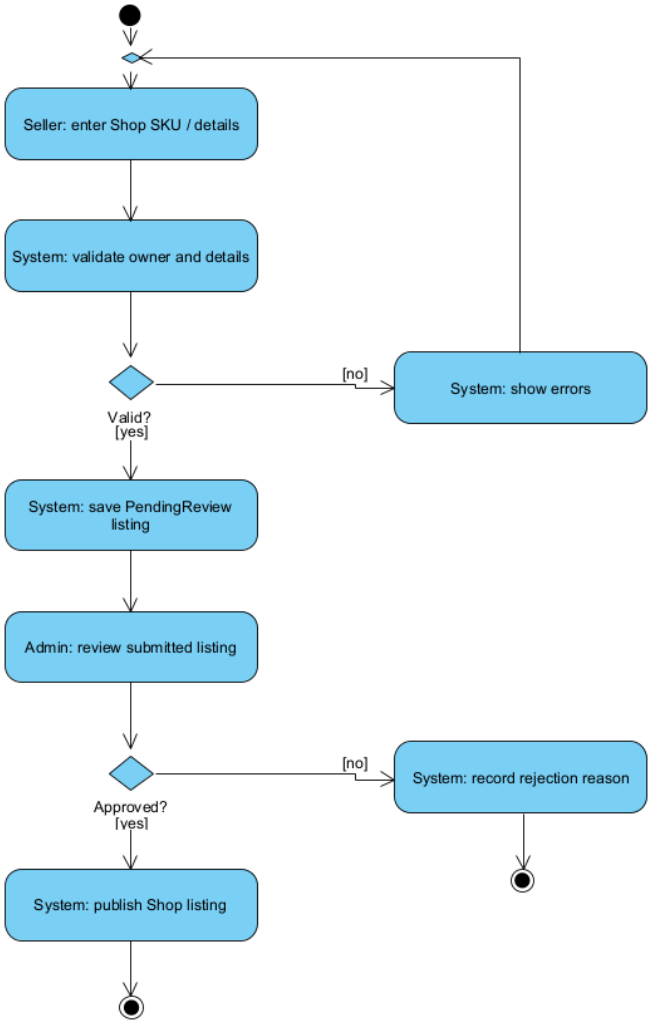


Figure 2.6: Manage Products and moderation activity diagram.

2.3.6. Manage Inventory

Table 2.7: Manage Inventory specification

Field	**Specification**
Use Case Name	Manage Inventory (UC14)
Primary Actor	Seller
Goal	The Seller records goods receipts, physical stock corrections, or verified return restocks for own-shop SKUs while protecting active order commitments.
Trigger	The Seller selects an SKU in their Shop inventory view to record a stock adjustment.
Preconditions	The Seller is authenticated and owns the Shop to which the target SKU belongs.
Postconditions	The SKU's availableStock is updated alongside an audit log recording the Seller, timestamp, quantity delta, and reason, while committedStock for active orders remains intact.

Field	Specification
Main Success Scenario	1. The Seller selects a target SKU within their Shop inventory list. 2. The system displays the SKU's current availableStock and the committedStock currently reserved for active orders. 3. The Seller enters a non-zero integer adjustment quantity and an audit reason (such as new stock receipt, warehouse count correction, or verified customer return). 4. The system verifies shop ownership, validates that the adjustment is a non-zero integer with a non-empty reason, and confirms that the resulting availableStock remains non-negative without cutting into committedStock. 5. When the adjustment represents a returned order restock, the system verifies that returnInspected is confirmed on the associated Refund record. 6. The system updates the SKU's availableStock, records the audit entry, and displays the updated inventory figures to the Seller.
Alternative Flows	1. Unauthorized SKU access: If the Seller attempts to adjust an SKU belonging to another Shop, the system rejects the request immediately. 2. Invalid quantity or missing reason: If the entered adjustment is zero, non-numeric, or missing an explanation, the system highlights the input error and leaves stock unchanged. 3. Reduction violates committed stock: If a negative adjustment would reduce availableStock below zero or encroach upon committedStock, the system rejects the reduction and displays the current committedStock constraint. 4. Unverified return restock: If an attempt is made to restock items from a refunded order before returnInspected is confirmed, the system blocks the stock increase until physical inspection is recorded. 5. Concurrent checkout race condition: If a Customer completes checkout on the same SKU while the Seller is submitting a stock reduction, the system re-evaluates the reduction against the newly updated committedStock before saving.
Exceptions	1. If saving the stock update and audit record fails, the database transaction rolls back completely and prompts the Seller to retry safely.
Business Rules	BR02, BR05, and BR11: Available stock must never become negative, committedStock is protected from manual overwrites, and returned goods increase stock only after confirmed physical inspection.
Priority	Must Have.

Figure 2.7 highlights how *Manage Inventory* checks shop ownership and active order commitments before updating available stock and recording the audit reason.

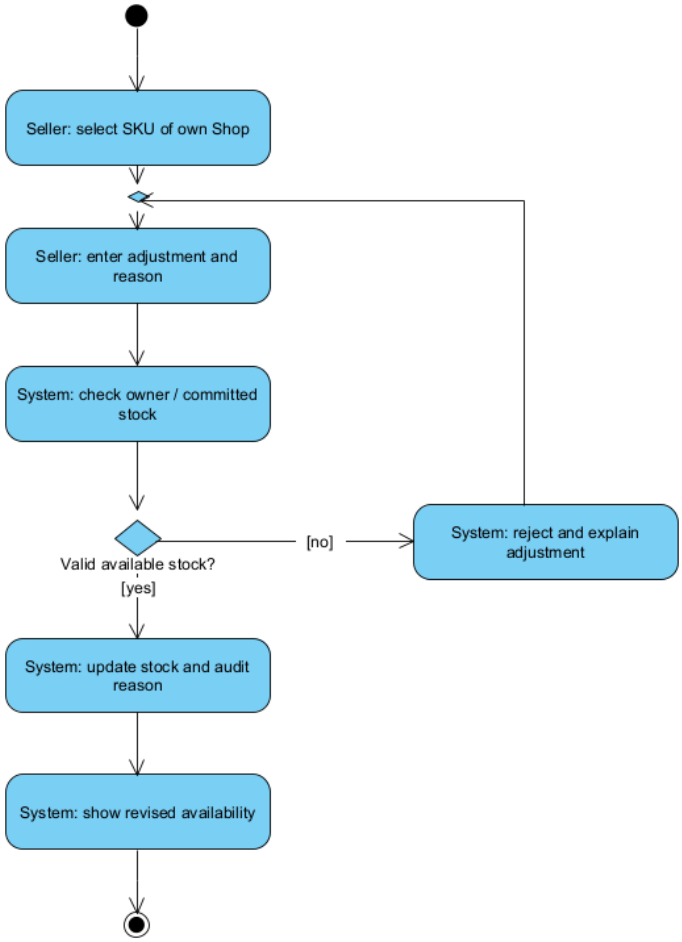


Figure 2.7: Manage Inventory activity diagram.

2.4. Supporting Processes and Transaction States

2.4.1. Order Fulfilment, Cancellation, and Post-Purchase Support

Beyond the six primary workflows modeled above, the remaining use cases govern order fulfilment, pre-handover cancellation, and post-purchase dispute resolution. In *Process Orders* (UC11), a Seller packs and hands over an order only after it belongs to their Shop and has reached the Placed state, which indicates either that online payment is verified as Paid or that a COD order is recorded with PendingCollection. Recording carrier handover creates the Shipment record with its tracking code and updatedAt timestamp and transitions the order to Shipped without deducting availableStock a second time.

Under *Cancel Order* (UC12), a Customer may cancel an order directly only while the order is in AwaitingPayment, Placed, or Processing prior to carrier handover. A valid pre-handover cancellation transitions the order to Cancelled and moves the reserved quantities from committedStock back to availableStock. Once the Seller has handed the parcel to the Shipping Provider, direct cancellation is blocked and any post-handover issue (such as damaged goods, wrong items, or delivery failure) must be submitted through *Resolve Support Case* (UC15). Each SupportCase links directly to the Order snapshot, records the buyer's reason and evidenceReference, captures the Seller's reply in sellerResponse, and stores Support Staff's final decision. When a claim is approved, the system creates a full-order Refund record for the collected order total, while any physical goods returned by the buyer are added back to availableStock only after the Seller confirms physical receipt and resale condition (setting returnInspected to true).

2.4.2. Financial Settlement and Period Reporting

Because dynamic shipping fee calculation is outside the system scope, an order's payable total equals the sum of its confirmed line-item amounts ($\sum \text{quantity} \times \text{unitPrice}$). When a full-order refund is approved, the customer is refunded the collected order total in full. Under *Manage Settlement* (UC16), an order becomes eligible for seller payout only when three conditions hold simultaneously: the parcel has reached the Delivered state, customer payment has been verified as collected (either online Paid or COD with codCollected set to true), and no open blocking SupportCase remains unresolved. Finance Staff record a Settlement en-

try capturing the grossAmount, platformFee, refundAmount, netAmount, status, and settledAt timestamp. If an exceptional dispute results in a refund after a payout was already disbursed, Finance Staff record an Adjusted settlement state rather than deleting historical payment records. Finally, in *Generate Reports* (UC17), reports generated for the Manager, Administrator, or Seller specify the reporting period and break down order totals, cancelled orders, collected funds, completed refunds, platform commission fees, and net seller payouts.

2.4.3. Transaction States

Physical parcel delivery, customer payment collection, dispute resolution, and seller settlement progress on independent schedules and are tracked through the six separate state lifecycles defined in Table 2.8.

Table 2.8: Transaction state definitions

Concept	Representative States	Meaning
Order	AwaitingPayment, Placed, Processing, Shipped, Delivered, Cancelled	Tracks purchase commitment and seller fulfilment stage. AwaitingPayment holds committedStock while waiting for online payment confirmation.
Payment	Pending, Paid, Failed, Unknown, PendingCollection	Tracks customer money collection. Unknown flags a gateway timeout awaiting Finance reconciliation, and PendingCollection marks COD orders before cash remittance.
Shipment	Preparing, InTransit, Delivered, Failed	Tracks physical parcel custody with the Shipping Provider independently of whether COD cash has been remitted.
SupportCase	Open, InReview, Resolved, Rejected	Tracks post-purchase complaint investigation from buyer submission and sellerResponse through staff decision.
Refund	Pending, Completed, Failed	Tracks the financial execution of an approved full-order refund and whether returned goods have been inspected (returnInspected).
Settlement	Pending, OnHold, Eligible, Paid, Adjusted	Tracks seller payout readiness, completed disbursement at settledAt, and post-payout refund adjustments.

When an order is placed with online payment, it enters AwaitingPayment with stock reserved until the payment outcome is verified, whereas a COD order enters Placed directly with Payment set to Pending-Collection. If an online payment times out in the Unknown state, the order remains in AwaitingPayment until Finance Staff reconcile the reference. Pre-handover cancellation from AwaitingPayment, Placed, or Processing transitions the order to Cancelled and releases the reserved stock once, while confirmed packing and carrier handover advance the order through Processing and Shipped to Delivered.

2.5. Domain Concepts

Fourteen domain concepts form the structural foundation of the analysis model. Each concept encapsulates both the state attributes required by the functional requirements and the conceptual operations that govern state transitions, as detailed in Table 2.9.

Table 2.9: Domain concepts, attributes, and conceptual responsibilities

Concept	Attributes and Key Operations	Conceptual Responsibility
Customer	customerId, name, contact, defaultAddress, status Operations: checkout(), cancelOrder()	Represents a buyer account that maintains a default delivery address, owns a cart, and places single-shop orders
Seller	sellerId, name, status Operations: registerShop(), manageListing()	Represents a merchant account that applies for and operates at most one Shop

Concept	Attributes and Key Operations	Conceptual Responsibility
Shop	shopId, name, pickupAddress, status, moderationReason, createdAt Operations: approve(), reject()	Represents a seller storefront with a carrier pickup address, moderation history, and single-shop orders
Product	productId, sku, name, description, category, unitPrice, availableStock, committedStock, status, rejectionReason Operations: commitStock(), releaseStock(), adjustStock()	Represents a sellable shop listing identified globally by productId and within its Shop by (shopId, sku), tracking live and committed stock
Cart	createdAt Operations: addItem(), calculateShopSubtotal()	Represents a Customer's temporary, multi-shop shopping container prior to checkout
CartItem	quantity Operation: updateQuantity()	Records the intended purchase quantity of one Product inside a Cart without holding stock
Order	orderCode, createdAt, status, total, addressSnapshot Operations: markPlaced(), cancel()	Represents a confirmed single-shop purchase with a locked payable total and delivery address snapshot
OrderItem	skuSnapshot, nameSnapshot, quantity, unitPrice Operation: getLineAmount()	Preserves the purchased quantity and fixed SKU, product name, and unit price agreed at checkout
Payment	method, amount, status, providerReference, codCollected, updatedAt Operations: recordResult(), reconcileUnknown()	Records the customer payment obligation, COD collection flag, timestamp, and verified outcome
Shipment	trackingCode, status, addressSnapshot, updatedAt Operations: recordHandover(), updateTracking()	Records carrier handover, parcel tracking code, delivery address snapshot, and status timestamp
SupportCase	caseId, reason, evidenceReference, sellerResponse, status, decision, createdAt Operations: submitSellerReply(), recordDecision()	Records a post-purchase complaint, buyer evidence, Seller response, and Support Staff decision
Refund	amount, status, reference, returnInspected, createdAt Operations: confirmReturnReceipt(), executeRefund()	Records an approved full-order customer refund and tracks whether returned goods passed physical inspection
Settlement	grossAmount, platformFee, refundAmount, netAmount, status, settledAt Operations: calculateNetEntitlement(), recordPayout()	Records seller merchandise value, platform fee, refund deduction, net payout, and settlement timestamp
Review	rating, comment, createdAt Operation: verifyDeliveredBuyer()	Records a verified buyer's rating and feedback comment for a delivered OrderItem

2.6. Cart, Order, and Checkout Analysis

2.6.1. Structural Distinction Between CartItem and OrderItem

A central structural decision in the domain model separates temporary shopping selections (Cart and CartItem) from binding purchase records (Order and OrderItem). Each Customer owns at most one active Cart, which may be empty or hold multiple CartItems drawn from different Shops. Because the desired

quantity is a property of a specific Product inside a specific Cart rather than a property of the Product itself, CartItem is modeled as a distinct class linking Cart and Product. While a CartItem exists, it references the live Product record to display current prices and availability grouped by Shop, leaving availableStock and committedStock untouched.

Once the Customer selects one shop group and completes checkout, the system creates an Order and one or more OrderItem instances for that Shop. Unlike a CartItem, an OrderItem is a permanent commercial record. While retaining a traceability link to the underlying Product record, OrderItem copies the SKU code, product name, and unit price into fixed snapshot attributes (skuSnapshot, nameSnapshot, and unitPrice), and Order copies the confirmed delivery address into addressSnapshot alongside the payable total. Consequently, if a Seller later changes a product's price, edits its description, or discontinues the listing, or if a Customer updates their defaultAddress on their profile, historical orders remain completely intact for fulfilment, dispute resolution, and financial auditing. The structural differences between CartItem and OrderItem are contrasted in Table 2.10.

Table 2.10: CartItem and OrderItem comparison

Aspect	**CartItem**	**OrderItem**
Business purpose	Records a temporary shopping intention	Records a confirmed purchase commitment
Shop scope	Belongs to a Customer Cart that can span multiple shops	Belongs to a single-shop Order between one Customer and one Shop
Lifecycle mutability	Freely editable or removable until checkout	Immutable once created at checkout acceptance
Pricing basis	Dynamically reads live Product unit price	Locks the confirmed transaction unit price snapshot
Product metadata	Dynamically reads live Product name and SKU	Stores fixed skuSnapshot and nameSnapshot values
Inventory impact	Leaves availableStock and committedStock unchanged	Moves purchased quantity from availableStock to committedStock
Effect of catalogue edits	Immediately reflects live price or status changes	Remains completely unaffected by later listing edits

2.6.2. Checkout Transformation

Checkout executes the state transformation that converts one selected shop group inside a Cart into a persisted Order and its constituent OrderItem records. Suppose a Customer's cart contains two units of an item from Shop A priced at 150,000 VND per unit and one unit of an item from Shop B priced at 300,000 VND. When the Customer invokes checkout(cart : Cart) : Order for Shop A, the system verifies that Shop A is active, the listing is published, and at least two units of availableStock exist. Upon confirmation, the system creates a new Order linked to Shop A with total equal to 300,000 VND and a locked addressSnapshot (copied from the Customer's defaultAddress or entered checkout address), creates the corresponding OrderItem storing quantity $= 2$ and unitPrice $= 150{,}000$ VND, invokes commitStock(2) on Shop A's Product (reducing availableStock by 2 and increasing committedStock by 2), and removes Shop A's CartItem from the cart. Shop B's CartItem remains in the cart for a future checkout.

2.7. Analysis Class Model

2.7.1. Core and After-Sales Class Diagrams

Figure 2.8 presents the UML analysis class diagram for the core shopping, ordering, payment, and shipment lifecycle, covering Customer, Cart, CartItem, Seller, Shop, Product, Order, OrderItem, Payment, and Shipment. Whole-part composition (filled diamond) connects Cart to CartItem and Order to OrderItem, expressing that line items cannot exist independently of their owning container.

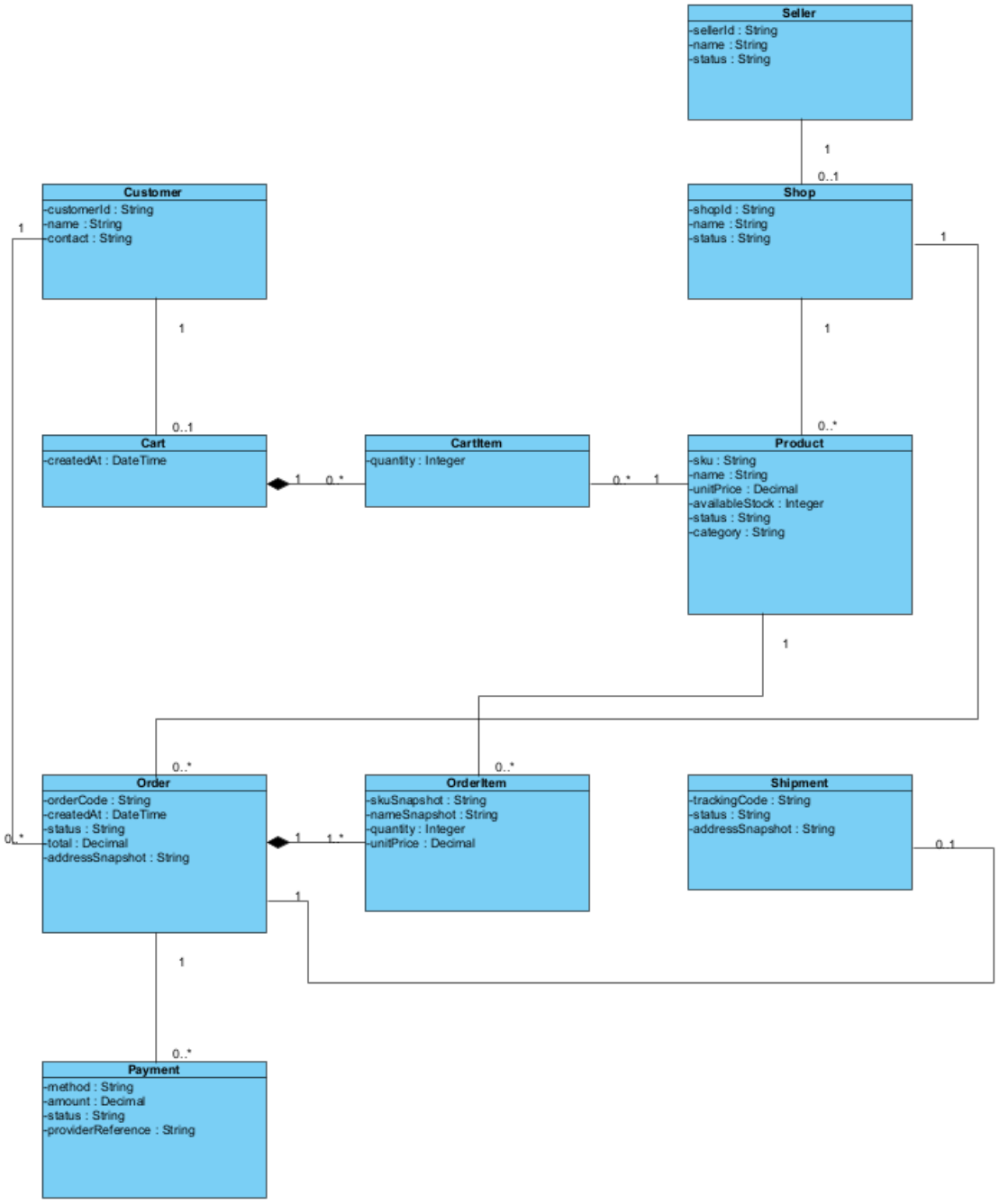


Figure 2.8: Core analysis class diagram for shopping, orders, payment, and delivery.

Figure 2.9 presents the analysis class diagram for post-purchase support, refunds, seller settlement, and buyer reviews, showing how SupportCase, Refund, and Settlement connect to Order and how Review connects to OrderItem. Modeling SupportCase, Refund, and Settlement as distinct classes with their own timestamps and status lifecycles ensures that post-delivery disputes, return inspections (returnInspected), customer refunds, and seller payouts are recorded without overwriting fields on the Order record.

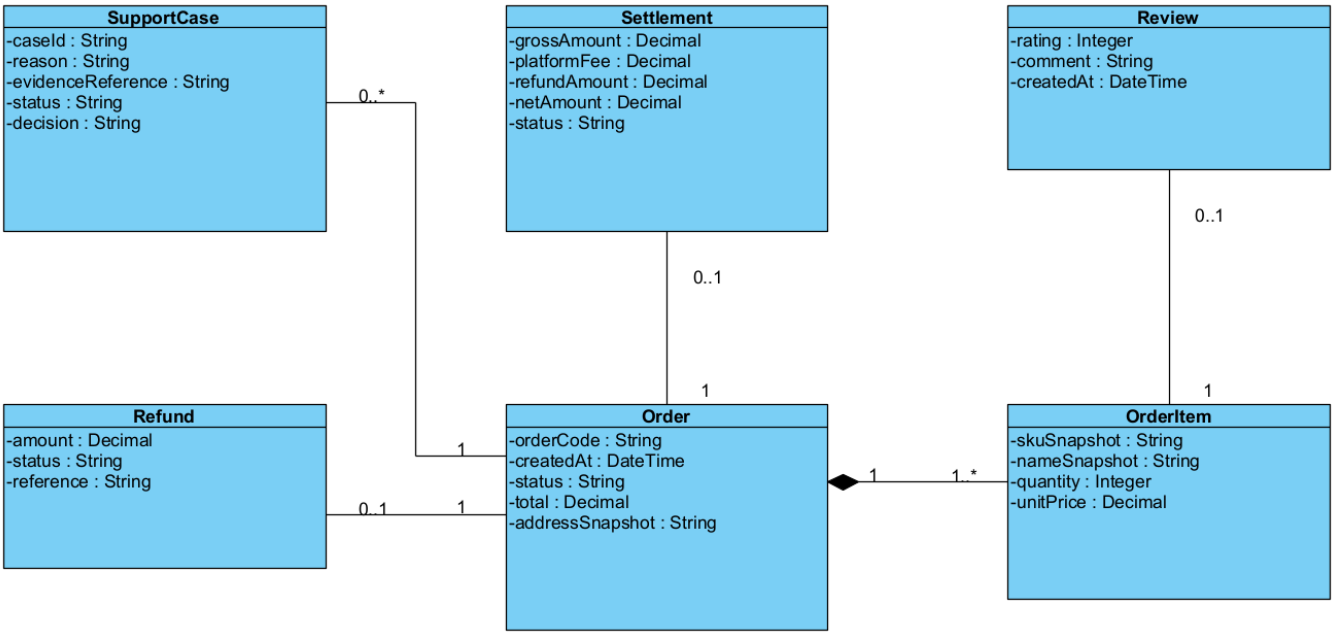


Figure 2.9: Analysis class diagram for support, refunds, reviews, and settlement.

2.7.2. Associations and Multiplicities

Every association and multiplicity in Figures 2.8 and 2.9 directly enforces a marketplace rule. The 1 to 0..1 multiplicity from Seller to Shop enforces the single-shop-per-seller policy, while the 1 to 0..* multiplicity from Shop to Order (paired with 1 to 1..* composition from Order to OrderItem) ensures that every order belongs to one selling Shop and contains at least one purchased item. Similarly, the 1 to 0..* multiplicity from Order to Payment allows an order to retain a failed payment attempt record alongside a subsequent verified collection, whereas the 1 to 0..1 multiplicities on Shipment, Refund, and Settlement enforce single-parcel shipping, full-order refunds, and single-settlement accounting per order. All fifteen associations and multiplicities are detailed in Table 2.11.

Table 2.11: Associations, multiplicities, and business meanings

Association	Multiplicity	Business Meaning
Customer to Cart	1 to 0..1	Each Customer owns at most one active shopping Cart at any time
Cart to CartItem	1 to 0..* (composition)	A Cart contains zero or more CartItems whose lifecycle is bound to that Cart
CartItem to Product	0..* to 1	Each CartItem references one live Product, and a Product may appear in many carts
Seller to Shop	1 to 0..1	Each Seller registers and operates at most one Shop, and every Shop has one owner
Shop to Product	1 to 0..*	Each Product listing belongs to one Shop under a shop-unique (shopId, sku) key
Customer to Order	1 to 0..*	Each Order is placed by one Customer, and a Customer may place many Orders over time
Shop to Order	1 to 0..*	Every Order is fulfilled by one Shop under the single-shop checkout policy
Order to OrderItem	1 to 1..* (composition)	Every Order contains at least one fixed OrderItem owned exclusively by that Order
OrderItem to Product	0..* to 1	Each OrderItem links to one Product (productId) while preserving its own transaction snapshots
Order to Payment	1 to 0..*	An Order logs all payment attempts, with at most one verified collection outcome
Order to Shipment	1 to 0..1	Each confirmed Order is dispatched in at most one carrier Shipment parcel

Association	Multiplicity	Business Meaning
Order to SupportCase	1 to 0..*	An Order may accumulate zero or more post-purchase SupportCases across its lifecycle
Order to Refund	1 to 0..1	An Order gives rise to at most one approved full-order Refund record
Order to Settlement	1 to 0..1	Each completed Order is accounted for in at most one seller Settlement record
OrderItem to Review	1 to 0..1	Each delivered OrderItem may receive at most one verified buyer Review

2.8. Business Rules

Fourteen cross-cutting business rules (BR01 to BR14) govern state transitions, access permissions, inventory reservations, and financial integrity across the use cases and class model, as listed in Table 2.12.

Table 2.12: Business rules

ID	Business Rule	Related Use Cases
BR01	Only published Products belonging to an active Shop can be displayed for sale or purchased.	UC04 to UC09
BR02	Cart and order quantities must be positive integers, and cart modifications never reserve stock.	UC08, UC09, UC14
BR03	Every Order and all of its constituent OrderItems must belong to a single selling Shop.	UC08, UC09
BR04	Checkout re-verifies live shop status, listing status, availableStock, and price, requiring buyer confirmation if a price changed.	UC09
BR05	Checkout moves purchased quantities from availableStock to committedStock once, pre-handover cancellation releases them once, and packing causes no second deduction.	UC09, UC11, UC12, UC14
BR06	OrderItem and Order snapshots remain unchanged when live product listings or customer defaultAddress profiles are later edited.	UC01, UC04, UC09
BR07	Online orders become eligible for seller packing only after verified payment, whereas COD orders proceed with pending collection.	UC09 to UC11
BR08	Unknown online payment timeouts must be reconciled by Finance Staff before retry, and a late payment on a cancelled order is refunded.	UC09, UC10, UC13
BR09	Customers may cancel an order directly only before the Seller hands the parcel over to the Shipping Provider.	UC12, UC15
BR10	Physical parcel delivery, COD cash collection (codCollected), and seller settlement payout are recorded and tracked as separate events.	UC11, UC13, UC16
BR11	Returned goods from a refunded order increase availableStock only after the Seller confirms physical receipt and resale condition (returnInspected = true).	UC14, UC15
BR12	Seller settlement eligibility requires confirmed parcel delivery, verified money collection, and no open blocking support case.	UC15, UC16
BR13	Only the Customer who purchased a delivered OrderItem may submit a review, limited to at most one review per order line.	UC18
BR14	Repeated provider callbacks or user requests carrying the same reference code are processed idempotently without duplicating money or stock changes.	UC09, UC10, UC13, UC14